



**Universidad
Tecnológica
del Perú**

FACULTAD DE INGENIERÍA

ESCUELA ACADÉMICA PROFESIONAL DE INGENIERÍA

Plan de Gestión de Capacidad y Rendimiento (ITIL)

CURSO

CURSO INTEGRADOR 2

DOCENTE

JONATHAN LOJA YOPLAC

TEMA

AVANCE DE PROYECTO FINAL 2

TÍTULO

**DESARROLLO E IMPLEMENTACIÓN DEL SISTEMA DE GESTIÓN DE PEDIDOS PARA EL RESTAURANTE
DRAGON DORADO**

INTEGRANTES GRUPO 5

**BURGA DÁVILA, ANDERSON DANIEL
GÁLVEZ LEZAMA, LUCILA JACKELINE
PEVES RAMIREZ, JANETH STEFANIE
SALGUERO BELLUZ, ANGEL
YAURI DE LA CRUZ, RENZO RODRIGO**

PERÚ

2026

1. Información del Servicio

Nombre del servicio	Sistema de Gestión de Pedidos Gastronómicos — Restaurante Dragón Dorado
Owner del servicio	Peves Ramirez Janeth Stefanie (Scrum Master)
Equipo técnico responsable	Burga Dávila, Anderson D. (Backend) · Salguero Belluz, Ángel (Frontend/UX) · Yauri de la Cruz, Renzo R. (QA/Cloud) · Gálvez Lezama, Lucila J. (Product Owner)
Nivel de criticidad	ALTO — sistema operacional en tiempo real que soporta la totalidad de las ventas del restaurante durante el horario comercial
Procesos de negocio soportados	Gestión de Atención Presencial (mapa de mesas, comanda digital) · Gestión y Control en Cocina (KDS) · Gestión de Atención para Delivery · Gestión y Control de Caja (apertura/cierre) · Gestión Administrativa (usuarios, roles, reportes)
Canales	Interfaz web SPA (React.js 18 + Tailwind CSS) accesible desde tablets y dispositivos móviles vía navegador · API REST (Spring Boot 3.5 / puerto 8080) · WebSocket STOMP para notificaciones en tiempo real entre salón y cocina
Dependencias internas	PostgreSQL 16 (contenedor Docker db_data) · Spring Boot 3.5.13 (contenedor andersonburga/dragon-rojo-backend:latest) · Spring Security 6 / JWT (Nimbus JOSE 10.9) · Spring WebSocket + STOMP
Dependencias externas	Cloudinary (almacenamiento de imágenes de productos) · VPS Contabo (infraestructura de producción) · Nginx (proxy inverso y servidor de frontend en puerto 8081) · SMTP para correos de activación de cuenta

2. Modelo de Demanda

Volumen de transacciones	Alta rotación: restaurante Chifa con atención continua en salón, cocina y delivery durante el horario comercial. Se estiman decenas de comandas por turno (mesas + delivery). Cada comanda genera múltiples eventos de escritura y lectura en tiempo real vía WebSocket
Usuarios concurrentes	~5–10 usuarios simultáneos en operación normal (mozos con tablet, 1–2 cocineros en KDS, 1 cajero, 1 administrador). Pico estimado: hasta 15 conexiones activas durante horas punta
Picos (horarios/eventos)	Hora punta de almuerzo (12:00–14:00) y cena (19:00–21:00): máxima concurrencia de comandas digitales y notificaciones WebSocket. El sistema debe mantener latencia ≤ 2 segundos en envío de comanda a cocina durante estos intervalos
Crecimiento proyectado	3 meses: estabilización y adopción total del personal (meta: 100% de uso activo en semana 3). 6 meses: incorporación del módulo de integración con apps de delivery (Rappi/PedidosYa) con incremento de volumen transaccional. 12 meses: posible expansión a nuevas sucursales, requiriendo aprovisionamiento de infraestructura VPS adicional o arquitectura multitenant
Supuestos de negocio	El restaurante cuenta con red Wi-Fi estable para la comunicación entre dispositivos. El personal operativo recibe capacitación básica para el uso

de la interfaz táctil. Los volúmenes actuales corresponden a un único local. La demanda puede escalar con la apertura de sucursales

3. Capacidad Actual

Componente	Capacidad	Uso actual	% Utilización
Aplicación (Backend Spring Boot)	Puerto 8080, JVM Amazon Corretto 21, Alpine Linux, perfil prod activo	Procesa solicitudes REST y WebSocket; HikariPool activo para conexiones BD	~30–50% en hora punta (estimado)
APIs REST	11 módulos: Auth, Users, Orders, Products, Categories, Clients, Drivers, Addresses, Tables, Cash, Dashboard	Endpoints activos en producción; Swagger UI disponible en /api/docs	Latencia promedio registrada: 323.83 ms (pruebas de login)
Base de datos (PostgreSQL 16)	Contenedor postgres:16-alpine, puerto externo 5434 → interno 5432; volumen Docker db_data; buffers: 195 buffers (1.2%)	Escucha en IPv4 (0.0.0.0) e IPv6; sincronización de archivos: 110 sync files; HikariPool conectado	Bajo (uso operacional inicial, sin saturación detectada)
Servidor web	Puerto 8081; archivos servidos desde /var/www/restaurant-frontend/dist; SPA con regla try_files	Active: active (running); estable durante todas las pruebas de carga	Memoria: 4.1 MB RAM (extremadamente eficiente)

Infraestructura (VPS Contabo)

CPU	Memoria RAM	Almacenamiento
VPS Contabo (especificaciones del plan contratado); contenedores en Alpine para mínimo overhead de CPU	Nginx estabilizado en 4.1 MB; Spring Boot (JVM) con HikariPool; total disponible según plan VPS contratado	Volumen Docker db_data para persistencia de PostgreSQL; backup diario con pg_dump; alerta por crecimiento de logs de contenedores detectada en Sprint 6

4. Evaluación de Brechas

Componente en riesgo	Cuello de botella identificado	Nivel de riesgo	Impacto en negocio
Logs de contenedores Docker	Crecimiento desmedido de logs durante operación continua; puede saturar el disco del VPS y ralentizar	MEDIO	Degradación del rendimiento del servidor y dificultad

	el análisis de eventos a largo plazo (detectado en Retrospectiva Sprint 6)		en auditoría de incidentes
Lazy loading de imágenes Cloudinary	Latencia inicial en carga de imágenes del menú en dispositivos móviles antes de activarse el mecanismo de lazy loading y caché del frontend (detectado Sprint 4)	BAJO	Experiencia de usuario levemente degradada en primera carga; no afecta operación crítica
Firewall VPS (redes híbridas IPv4/IPv6)	Dificultades de configuración en reglas de firewall para auditar limpiamente tráfico de conexiones móviles externas (detectado Sprint 6)	BAJO	Riesgo de monitoreo incompleto en conexiones externas; no afecta la disponibilidad del servicio
Almacenamiento externo (Backup S3)	El volumen db_data persiste correctamente en Docker pero no existe automatización del volcado diario (pg_dump) hacia almacenamiento externo tipo S3 en producción	ALTO	Riesgo de pérdida de datos ante falla catastrófica del VPS; compromete la continuidad del negocio

5. Gestión de Rendimiento

KPIs definidos

ID	Indicador	Descripción	Meta	Línea base
KPI-01	Tasa de error en pedidos	Pedidos con datos incorrectos / total pedidos del día. Medición: correcciones registradas en sistema	< 0.5% al mes 3	~15% (comandas físicas)
KPI-02	Tiempo promedio de atención (salón)	Desde que el cliente ordena hasta que el plato llega a mesa. Timestamp comanda enviada vs. estado 'Listo para servir'	≤ 12 min (hora punta)	~18 min estimado
KPI-03	Tiempo de despacho delivery	Desde registro del pedido hasta que el motorizado sale del local. Timestamp registro vs. estado 'Despachado'	≤ 20 min	Sin referencia digital previa
KPI-04	Tiempo de cierre de caja	Desde apertura del proceso de cuadre en el sistema hasta su	≤ 10 min	25 min (proceso manual)

		finalización. Registro inicio y fin del módulo de caja		
KPI-05	Satisfacción del usuario interno	Encuesta post-implementación a mozos, cocineros y cajeros (escala 1–5). Frecuencia: mensual (primeros 3 meses)	≥ 4.2 / 5	Sin referencia previa
KPI-06	Tasa de adopción del sistema	% del personal que usa el sistema activamente (≥ 1 acción por turno) Medición: log de sesiones activas	80% sem1 → 100% sem3	0% (proceso manual)

SLA — Disponibilidad	≥ 99.9% de uptime durante horario comercial
SLA — Latencia comanda	≤ 2 segundos desde envío al servidor hasta recepción en monitor KDS (WebSocket)
SLA — Generación de reportes	≤ 5 segundos para carga del dashboard gerencial
SLA — Resolución de incidentes	≤ 4 horas para fallas críticas
Estado actual	Backend Spring Boot: 323.83 ms promedio en solicitudes críticas (login / verificación JWT) — cumple holgadamente SLA. Nginx: 4.1 MB RAM, estado active (running) sin interrupciones detectadas
Alertas recientes	Crecimiento de logs Docker durante operación continua (Sprint 6). Variables de entorno en puertos PostgreSQL durante primer despliegue (5434 → 5432) — resuelto con reconfiguración de docker-compose.yml
Herramientas de monitoreo	systemctl status nginx (estado del servidor web) · docker logs (auditoría de transacciones del backend) · pg_dump (backup manual de BD) · Planificado: Prometheus + Grafana para métricas automáticas de CPU/RAM

6. Plan de Capacidad

Necesidad futura	Acción planificada	Responsable	Fecha estimada
Automatización de backups a almacenamiento externo	Implementar script cron-job con pg_dump + envío a Bucket S3 de AWS para volcado diario seguro		Sprint 7 (corto plazo)
Rotación de logs de contenedores Docker	Configurar política logrotate en Docker daemon: máximo 10 MB por archivo, retención de 3 archivos por contenedor		Sprint 7 (corto plazo)

Pipeline CI/CD automatizado	Implementar GitHub Actions para pruebas JUnit 5 y despliegue automático al VPS tras cada merge aprobado en rama main		Sprint 7–8
Monitoreo proactivo de rendimiento	Integrar Prometheus + Grafana para alertas automáticas cuando CPU o RAM de PostgreSQL supere el 80% durante horas punta		Sprint 8–9
Escalamiento para nuevas sucursales	Aprovisionamiento de VPS adicional o arquitectura multitenant; base de datos aislada o compartida con particionamiento por sucursal		12 meses (según demanda)
Integración con apps de delivery (Rappi/PedidosYa)	Implementación de APIs de terceros para automatizar registro de pedidos externos (actualmente proceso manual)		6 meses

7. Plan de Mejora

Problema identificado	Acción de mejora	Prioridad	Fecha compromiso
Crecimiento descontrolado de logs Docker en disco del VPS	Implementar logrotate con límite 10 MB / 3 archivos por contenedor para evitar saturación del almacenamiento	ALTA	Sprint 7
Backup de BD sin automatización en almacenamiento externo	Script cron-job para pg_dump diario y transferencia segura a S3 de AWS; validar integridad tras cada volcado	ALTA	Sprint 7
Despliegue manual propenso a errores de configuración	GitHub Actions: pipeline CI/CD con etapas de build, test (JUnit 5) y deploy automático al VPS tras merge en main	ALTA	Sprint 7–8
Área táctil pequeña en comanda digital (tablets mozos)	Ampliar padding de tarjetas de platos en App.tsx y Tailwind CSS del mapa de mesas y comanda para reducir errores accidentales de selección	MEDIA	Sprint 7
Ausencia de monitoreo proactivo automatizado	Integrar Prometheus + Grafana con alertas al canal técnico cuando CPU/RAM del contenedor PostgreSQL supere 80%	MEDIA	Sprint 8–9
Reglas de firewall para tráfico móvil IPv4/IPv6 sin estandarizar	Revisar y documentar reglas de firewall del VPS para auditar limpiamente	BAJA	Sprint 8

	conexiones de dispositivos externos	
--	-------------------------------------	--

8. Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Caída del servidor backend o frontend (VPS)	BAJO	ALTO	Monitoreo de disponibilidad con alertas automáticas; reinicio de contenedores y verificación de conexión BD antes de reanudar operaciones
Fallo o corrupción de base de datos PostgreSQL	BAJO	ALTO	Backups automáticos diarios con retención mínima de 7 días; restauración desde último backup disponible con verificación de integridad
Corte de conexión WebSocket (salón ↔ cocina KDS)	MEDIO	ALTO	Pruebas de estabilidad en entorno de producción; el personal puede consultar pedidos directamente desde la interfaz web mientras se restablece la conexión
Robo de refresh token (suplantación de sesión)	MEDIO	ALTO	Transmisión siempre sobre HTTPS; tiempo de expiración corto del access token (15 min por defecto); revocación del refresh token en tabla refresh_tokens ante incidente
Error durante despliegue cloud (AWS/VPS) con datos inconsistentes	MEDIO	ALTO	Despliegue primero en entorno de pruebas; validar todos los módulos antes de pasar a producción; revertir a versión estable anterior si hay errores
Indisponibilidad de Cloudinary (bloqueo de actualización de menú)	BAJO	MEDIO	Gestionar actualizaciones del menú fuera de horarios de alta demanda; posponer carga de imágenes hasta que el servicio se restablezca; platos existentes siguen operativos
Saturación del almacenamiento por crecimiento de logs Docker	MEDIO	MEDIO	Implementar política logrotate; límite 10 MB / 3 archivos por contenedor; monitoreo periódico del espacio en disco del VPS

9. Pruebas de Capacidad

Tipo de pruebas realizadas	Pruebas funcionales: Gestión de Acceso de Usuario, Gestión de Clientes, Gestión de Mesas, Gestión de Órdenes, Gestión de Productos, Gestión de Cajas. Pruebas no funcionales: RNF01 Usabilidad (interfaz responsiva móvil), RNF02 Disponibilidad (uptime horario comercial), RNF03 Rendimiento (latencia de login), RNF04 Seguridad (cifrado de contraseñas / JWT), RNF05 Escalabilidad (arquitectura Docker + VPS).
Resultados obtenidos	RNF03 Rendimiento: tiempo de respuesta promedio de 323.83 ms en solicitudes críticas de login y verificación de token JWT — cumple

Punto de saturación	holgadamente el SLA de ≤ 2 segundos para envío de comanda. Nginx: Memory 4.1 MB RAM, estado active (running), sin interrupciones durante pruebas de carga. PostgreSQL: 195 buffers inicializados (1.2%), 110 sync files, sin cuellos de botella en disco. Despliegue Docker exitoso en VPS con transición limpia desde entorno de desarrollo (perfil prod activo) No alcanzado en pruebas actuales. Los resultados muestran que la infraestructura opera con margen amplio bajo la carga actual (~5–15 usuarios concurrentes). El punto de saturación deberá determinarse con pruebas de estrés formales usando herramientas como JMeter o k6 en sprints posteriores, simulando el escenario de máxima concurrencia (hora punta + delivery simultáneo)
Conclusiones	El stack tecnológico seleccionado (Spring Boot 3.5 / JVM / Docker Alpine / Nginx / PostgreSQL 16) demostró estabilidad y eficiencia en el entorno de producción VPS. Las métricas de rendimiento cumplen todos los SLA definidos. La integración del ciclo JWT stateless y el cifrado BCrypt fueron validados correctamente. Se identificaron áreas de mejora en la gestión de logs, automatización de backups y UX táctil que no comprometen la operatividad actual pero deben abordarse proactivamente en sprints futuros

10. Gobierno

Frecuencia de revisión	Operativa: al finalizar cada sprint (cada 3 semanas) en la sesión de Retrospectiva Scrum. Táctica: mensual durante los primeros 3 meses post-implementación para evaluar adopción (KPI-05, KPI-06). Estratégica: trimestral o ante eventos de escalamiento (nuevas sucursales, picos de demanda inusuales)
Indicadores de control	% de uptime del sistema (meta $\geq 99.9\%$) · Latencia promedio de envío de comanda a cocina (meta ≤ 2 s) · % de cumplimiento de SLA de incidentes (tickets atendidos dentro del plazo / total tickets) · Tiempo promedio de asignación de tickets · Tasa de adopción del sistema (meta 100% semana 3) · Satisfacción del usuario interno (meta $\geq 4.2/5$) · Tiempo de cierre de caja (meta ≤ 10 min) · Tasa de error en pedidos (meta $< 0.5\%$)
Responsables	asdasdasd. — QA/Cloud: monitoreo de infraestructura, logs y rendimiento. asdasdasd — Backend: integridad de la aplicación y base de datos. asdasdasdasdasd — Scrum Master: coordinación de revisiones, seguimiento de KPIs y comunicación con el Product Owner. asdasdasdasd — Product Owner: validación de cumplimiento de SLA y priorización del plan de mejora
Relación con ITSM	Incidentes: gestionados mediante el Catálogo de Solicitudes de Servicio (fallas críticas con SLA ≤ 4 horas); registro obligatorio en Excel de Requerimientos con número de ticket correlativo. Cambios: gestionados según el Proceso de Gestión de Cambios documentado en el Capítulo VIII; aprobación del Scrum Master antes de cada despliegue; validación previa en entorno de pruebas. Problemas: identificados en retrospectivas de sprint (Sprint 4 y Sprint 6); acciones de mejora incorporadas al backlog del siguiente sprint para cierre formal